

Practical File
of
Introduction to Web Technologies (25CSE0101)
Batch-2025
Bachelor of Engineering (CSE)



Submitted By
Tanish Singla

Roll No: 2510990736
Group No -5, Sem: 1st, BE (CSE)
Chitkara University, Punjab, 140401
India

Submitted To

Dr. Gaganpreet Kaur

Chitkara University, Punjab, 140401
India

Department of
Computer Science and Engineering,
Chitkara University School of Engineering and Technology,
Chitkara University, Punjab, India

Index

Sr. No.	Experiment	Page No.
1	Explain Client–Server Architecture and Observation of HTTP Requests and Responses using Browser Developer Tools	3
2	Create a basic blog webpage that includes a header, paragraph text, and both ordered and unordered lists	4-5
3	Insert an image and add hyperlinks, including both internal (within the same website) and external (to other websites) links	6-7
4	Structure the blog content using semantic HTML tags such as <header>, <main>, and <footer> for better readability and SEO	8-9
5	Design a contact form that includes a variety of input elements such as text fields, radio buttons, checkboxes, dropdown selections, and a text area	10-11
6	Construct a product listing section using an HTML table, incorporating borders, cell spacing, and alignment features for proper layout	12-13
7	Enhance the webpage with a favicon and appropriate meta tags to improve SEO and user experience	14-15
8	Apply internal and external CSS styles to format the blog page, demonstrating control over presentation	16-18
9	Utilize class selectors, ID selectors, and element selectors to style specific components of the webpage. Demonstrate CSS rule precedence and specificity by combining inline, internal, and external styles in one project	19-21
10	Use CSS to apply text formatting, background colors, borders, padding, and margins to different webpage elements	22-24
11	Illustrate the CSS box model by styling elements with borders and adjusting their spacing properties	25-27
12	Implement a variety of CSS units such as em, rem, %, and px to observe how different units impact design	28-30
13	Design a responsive navigation bar using Flexbox properties such as justify-content and align-items	31-33
14	Implement media queries to ensure the layout adapts correctly on different screen sizes like mobile and desktop	34-35
15	Apply responsive design techniques using properties like min-width, max-width, and include the viewport meta tag for mobile optimization	36-37
16	Create interactive hover effects using pseudo-classes like :hover and :active on various element	38
17	Use pseudo-elements ::before and ::after to insert decorative or functional content without modifying HTML	39
18	Develop a simple CSS animation (e.g., fading in/out an image or transforming a button on hover)	40
19	Develop a Web Page Layout using Bootstrap Grid System and Utility Classes	41-42
20	Design and deploy a Static Website to GitHub Pages	43-49

EXPERIMENT 01

AIM - Explain Client–Server Architecture and Observation of HTTP Requests and Responses using Browser Developer Tools.

Client-Server Architecture is a computing model where the client (e.g., a user's web browser or application) requests resources or services from a centralized server, which processes the request and sends back the response, enabling efficient distribution of tasks and data management. This architecture separates concerns: the client handles user interface and input, while the server manages data storage, processing, and security.

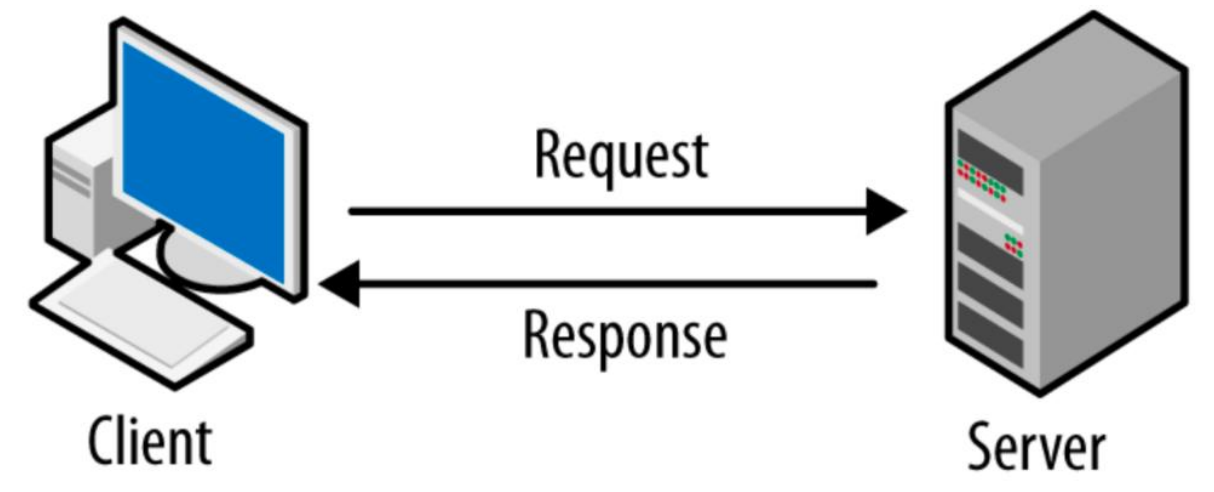


Figure 1 - Client Server Architecture

In the diagram, the client initiates communication by sending a request to the server, which then responds with the required data or action result.

What are HTTP Requests and Responses?

HTTP Request: This is a message sent by your browser to a web server when you go to a website (e.g., you are requesting an HTML page, CSS, JavaScript, images, APIs).

HTTP Response: The server's reply to the request (status code, headers, and the actual content).

How to See Them Using Developer Tools in Your Browser

Open Developer Tools (F12 or Ctrl+Shift+I).

Open the Network tab and refresh the Page.

Click on the refresh/reload button or Ctrl+R.

Explore HTTP Requests -You should see a list of all resources (HTML, CSS, JS, images, fonts, API calls).

EXPERIMENT 02

AIM - Create a basic blog webpage that includes a header, paragraph text, and both ordered and unordered lists.

For this task, we created a simple blog webpage using basic HTML tags like header, paragraph text, and both ordered and unordered lists.

Header Section - We began the webpage by using the <header> tag, which defines the top section of the blog. Inside the header, we placed an <h1> element to display the blog's main title. The header helps readers immediately understand what the page is about, and it also improves SEO.

Paragraph and Lists - Inside the article, we wrote a paragraph using the <p> tag to introduce the blog. We also included two types of lists. An **ordered list ()** was used to show steps in a proper sequence, such as the steps to start blogging. An **unordered list ()** was used for general items, like resources. Each item in both lists was defined using the tag.

PROGRAM-

```
<!DOCTYPE html>
<html>
<head>
  <title>My Simple Blog</title>
</head>
<body>

  <h1>Welcome to My Blog</h1>

  <p>
    This is a simple blog page created using basic HTML.
    It includes some text, an ordered list, and an unordered list.
  </p>

  <h2>My Favorite Hobbies</h2>
  <ul>
    <li>Reading</li>
    <li>Cycling</li>
    <li>Photography</li>
  </ul>

  <h2>Steps to Start a Blog</h2>
  <ol>
    <li>Choose a topic</li>
    <li>Write your first post</li>
    <li>Publish online</li>
  </ol>

</body>
</html>
```

Figure 2 - Code Snippet

OUTPUT-

Welcome to My Blog

This is a simple blog page created using basic HTML. It includes some text, an ordered list, and an unordered list.

My Favorite Hobbies

- Reading
- Cycling
- Photography

Steps to Start a Blog

1. Choose a topic
2. Write your first post
3. Publish online

Figure 3 - Final Web Page Layout

EXPERIMENT – 03

AIM - Insert an image and add hyperlinks, including both internal (within the same website) and external (to other websites) links.

For this task, we extended our basic blog webpage by adding an image and hyperlinks using standard HTML tags.

Image Section – We inserted an image using the tag. The src attribute specifies the path/URL of the image, while the alt attribute provides alternative text if the image cannot be displayed.

Internal Link – We created an internal hyperlink using the <a> tag that points to another page within the same website. This allows users to navigate between different sections of the site easily.

External Link – We added an external hyperlink using the <a> tag that points to an outside website (e.g., Wikipedia). The target="_blank" attribute ensures the link opens in a new tab.

PROGRAM-

```
<!DOCTYPE html>
<html>
<head>
  <title>My Simple Blog</title>
</head>
<body>
  <h1>Welcome to My Blog</h1>
  <p>
    This is a simple blog page created using basic HTML.
    It includes text, an ordered & unordered list, image , an internal & external link.
  </p>

  <h2>My Favorite Hobbies</h2>
  <ul>
    <li>Reading</li>
    <li>Cycling</li>
    <li>Photography</li>
  </ul>

  <h2>Steps to Start a Blog</h2>
  <ol>
    <li>Choose a topic</li>
    <li>Write your first post</li>
    <li>Publish online</li>
  </ol>
  <h2>Useful Links</h2>
  <p>
    Visit my <a href="about.html">About Page</a> (internal link). <br>
    Check out <a href="https://www.wikipedia.org" target="_blank">Wikipedia</a> (external link)
  </p>
</body>
</html>
```

Figure 4 - Code Snippet

Welcome to My Blog

This is a simple blog page created using basic HTML.

It includes text, an ordered & unordered list, an image, and both internal & external links.



My Favorite Hobbies

- Reading
- Cycling
- Photography

Steps to Start a Blog

1. Choose a topic
2. Write your first post
3. Publish online

Useful Links

Visit my [About Page](#) (internal link).

Check out [Wikipedia](#) (external link).

Figure 5 - Final Web Page Layout

EXPERIMENT 04

AIM - Structure the blog content using semantic HTML tags such as <header>, <main>, and <footer> for better readability and SEO

For this task, we improved the structure of our basic blog webpage by organizing the content using **semantic HTML tags**. These tags help search engines and assistive technologies understand the layout and purpose of each section of the webpage.

Header Section – We used the <header> tag to define the top section of the blog. It contains the main title or heading of the page, helping users quickly understand the topic of the site.

Main Section – The <main> tag is used to wrap the primary content of the blog, such as paragraphs, lists, images, and hyperlinks. It represents the central information that the page is meant to display.

Footer Section – The <footer> tag is used to define the bottom section of the webpage. It may include copyright information, contact details, or useful links, providing closure to the content.

PROGRAM-

```
<!DOCTYPE html>
<head>
  <title>My Simple Blog</title>
</head>
<body>
<header>
  <h1>Welcome to My Blog</h1>
  <p>
    This is a simple blog page created using basic HTML.
    It includes text, an ordered & unordered list, an image, and internal & external links.
  </p>
</header>
<main>

  <h2>My Favorite Hobbies</h2>
  <ul>
    <li>Reading</li>
    <li>Cycling</li>
    <li>Photography</li>
  </ul>
  <h2>Steps to Start a Blog</h2>
  <ol>
    <li>Choose a topic</li>
    <li>Write your first post</li>
    <li>Publish online</li>
  </ol>
  <h2>Useful Links</h2>
  <p>
    Visit my <a href="about.html">About Page</a> (internal link). <br>
    Check out <a href="https://www.wikipedia.org" target="_blank">Wikipedia</a> (external link).
  </p>
</main>
```



```
<footer>
  <p>&copy; 2025 My Blog | All Rights Reserved</p>
</footer>

</body>
</html>
```

Figure 6 - Code Snippet

OUTPUT-

Welcome to My Blog

This is a simple blog page created using basic HTML. It includes text, an ordered & unordered list, an image, and both internal & external links.



My Favorite Hobbies

- Reading
- Cycling
- Photography

Steps to Start a Blog

1. Choose a topic
2. Write your first post
3. Publish online

Useful Links

Visit my [About Page](#) (internal link).
Check out [Wikipedia](#) (external link).

© 2025 My Blog | All Rights Reserved

Figure 7 - Final Web Page Layout

EXPERIMENT 05

AIM - Design a contact form that includes a variety of input elements such as text fields, radio buttons, checkboxes, dropdown selections, and a text area

For this task, we created a **contact form** using various HTML form elements to collect user input efficiently. The `<form>` tag defines the overall form structure, while different input elements allow users to provide information in multiple ways.

Text Fields: We used `<input type="text">` and `<input type="email">` to collect the user's name and email address. These allow users to enter short text-based information.

Radio Buttons: The `<input type="radio">` elements were used to let the user choose one option from a predefined set (e.g., gender).

Checkboxes: The `<input type="checkbox">` tag lets users select multiple interests or preferences at once.

Dropdown Selection: The `<select>` and `<option>` tags were used to create a dropdown menu that allows users to choose one value from a list (e.g., country or subject).

Text Area: The `<textarea>` tag provides a larger input area for users to write detailed messages or comments.

Submit Button: Finally, the `<input type="submit">` element was added to send the form data.

PROGRAM-

```
<!DOCTYPE html>
<html>
<head>
  <title>Contact Form</title>
</head>
<body>
  <h1>Contact Us</h1>

  <form action="#" method="post">
    <label for="name">Full Name:</label><br>
    <input type="text" id="name" name="name" placeholder="Enter your name" required><br><br>

    <label for="email">Email:</label><br>
    <input type="email" id="email" name="email" placeholder="Enter your email" required><br><br>

    <label>Gender:</label><br>
    <input type="radio" id="male" name="gender" value="Male">
    <label for="male">Male</label>
    <input type="radio" id="female" name="gender" value="Female">
    <label for="female">Female</label><br><br>

    <label>Interests:</label><br>
    <input type="checkbox" id="reading" name="interest" value="Reading">
    <label for="reading">Reading</label>
    <input type="checkbox" id="traveling" name="interest" value="Traveling">
    <label for="traveling">Traveling</label>
    <input type="checkbox" id="music" name="interest" value="Music">
    <label for="music">Music</label><br><br>
```

```

<label for="subject">Subject:</label><br>
<select id="subject" name="subject">
  <option value="">--Select--</option>
  <option value="feedback">Feedback</option>
  <option value="support">Support</option>
  <option value="inquiry">General Inquiry</option>
</select><br><br>

<label for="message">Your Message:</label><br>
<textarea id="message" name="message" rows="4" cols="40" placeholder="Write your message
here..."></textarea><br><br>
<input type="submit" value="Submit">
</form>
</body>
</html>

```

Figure 8 - Code Snippet

OUTPUT-

Contact Us

Full Name:

Email:

Gender:

☐ Male ☐ Female

Interests:

☐ Reading ☐ Traveling ☐ Music

Subject:

Your Message:

Figure 9 - Final Web Page Layout

EXPERIMENT – 06

AIM - Construct a product listing section using an HTML table, incorporating borders, cell spacing, and alignment features for proper layout

For this task, we created a **product listing section** using an HTML <table>. Tables are ideal for displaying structured data like products, prices, and descriptions.

Table Creation: The <table> tag defines the table container.

Rows and Columns: <tr> defines a row, while <th> and <td> define header and data cells, respectively.

Borders: The border attribute is used to make the table visually distinct.

Cell Spacing & Padding: The cell spacing and cellpadding attributes control the space between and inside table cells.

Alignment: The align attribute allows content to be aligned within cells, making the table more readable.

PROGRAM-

```
<!DOCTYPE html>
<html>
<head>
  <title>Product Listing</title>
</head>
<body>

  <h1>Our Products</h1>

  <table border="1" cellspacing="5" cellpadding="10" align="left">
    <tr>
      <th>Product ID</th>
      <th>Product Name</th>
      <th>Category</th>
      <th>Price ($)</th>
      <th>Stock</th>
    </tr>
    <tr>
      <td>P001</td>
      <td>Wireless Mouse</td>
      <td>Electronics</td>
      <td>25</td>
      <td>Available</td>
    </tr>
    <tr>
      <td>P002</td>
      <td>Bluetooth Headphones</td>
      <td>Electronics</td>
      <td>50</td>
      <td>Out of Stock</td>
    </tr>
  </table>
```

```
<td>P003</td>
<td>Cotton T-Shirt</td>
<td>Apparel</td>
<td>15</td>
<td>Available</td>
</tr>
<tr>
<td>P004</td>
<td>Running Shoes</td>
<td>Footwear</td>
<td>60</td>
<td>Available</td>
</tr>
</table>

</body>
</html>
```

Figure 10 - Code Snippet

OUTPUT-

Our Products

Product ID	Product Name	Category	Price (\$)	Stock
P001	Wireless Mouse	Electronics	25	Available
P002	Bluetooth Headphones	Electronics	50	Out of Stock
P003	Cotton T-Shirt	Apparel	15	Available
P004	Running Shoes	Footwear	60	Available

Figure 11 - Final Web Page Layout

EXPERIMENT – 07

AIM - Enhance the webpage with a favicon and appropriate meta tags to improve SEO and user experience

For this task, we improved the webpage by adding **meta tags** and a **favicon**, which are important for both **SEO** and **user experience**.

Favicon: A favicon is a small icon that appears in the browser tab next to the page title. It helps users quickly identify your site and improves branding. We add it using the `<link rel="icon">` tag, specifying the path or URL to the icon image.

Meta Tags: Meta tags provide information about the webpage to search engines and browsers. Common meta tags include:

`<meta name="description" content="...">` → Gives a summary of the page for search engine results.

`<meta name="keywords" content="...">` → Helps define relevant keywords

`<meta name="viewport" content="width=device-width, initial-scale=1.0">` → Ensures the page is responsive on mobile devices.

PROGRAM-

```
<!DOCTYPE html>
<head>
  <title>My Simple Blog</title>

  <link rel="icon" href="favicon.ico" type="image/x-icon">

  <meta name="description" content="A simple blog page.">
  <meta name="keywords" content="blog, hobbies, web development, tutorials">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>

  <header>
    <h1>Welcome to My Blog</h1>
    <p>This is a simple blog page created using HTML, enhanced with a favicon and meta tags for better SEO.</p>
  </header>

  <main>
    <p>Blog content goes here...</p>
  </main>

  <footer>
    <p>&copy; 2025 My Simple Blog | All Rights Reserved</p>
  </footer>

</body>
</html>
```

Figure 12 - Code Snippet

OUTPUT-



Figure 13 - Final Web Page Layout

EXPERIMENT – 08

AIM – Apply internal and external CSS styles to format the blog page, demonstrating control over presentation

In this experiment, we enhanced our blog webpage by applying **internal** and **external** CSS styles to improve its visual presentation and layout.

- **Internal CSS** – This style is defined inside the <style> tag within the <head> section of the HTML document. It is useful for adding styles that apply only to that specific page.
Example: Setting background color, font styles, or margins for the page.
- **External CSS** – This style is written in a separate .css file and linked to the HTML using the <link> tag. It allows us to maintain consistent styling across multiple pages and makes updates easier.

By combining internal and external CSS, we can achieve a clean, attractive, and consistent design for our blog while keeping our HTML structured and readable.

PROGRAM-

```
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>My Blog</title>
<style>
  body {
    font-family: Arial, sans-serif;
    margin: 20px;
  }
  header {
    background: blueviolet;
    color: red;
    text-align: center;
    padding: 10px;
  }
  nav {
    text-align: center;
    margin: 10px 0;
  }
  nav a {
    margin: 0 10px;
    text-decoration: none;
    color: black;
  }
  section {
    margin: 20px 0;
  }
  img {
    width: 100%;
    max-width: 600px;
    display: block;
    margin: auto;
  }
```



```

}
footer {
  text-align: center;
  background: black;
  color: white;
  padding: 10px;
  margin-top: 20px;
}
</style>
</head>
<body>

<header>
  <h1>My Simple Blog</h1>
</header>

<nav>
  <a href="#about">About Me</a>
  <a href="https://www.chitkara.edu.in" target="_blank">College Website</a>
</nav>

<section id="about">
  <h2>About Me</h2>
  <p>Hello! I am Tanish Singla . This is my blog.</p>

  <h3>My Hobbies</h3>
  <ul>
    <li>Reading</li>
    <li>Travelling</li>
    <li>Coding</li>
    <li>Gaming</li>
  </ul>

  <h3>My Routine</h3>
  <ul>
    <li>Morning: Exercise</li>
    <li>Day: Study/Work</li>
    <li>Night: Relax</li>
  </ul>
</section>
<section>
  <h2>My Favorite Place</h2>
  
  <p>I love this place because it is the end.</p>
</section>
<footer>
  <p>Thanks for visiting!</p>
</footer>
</body>
</html>

```

Figure 14 - Code Snippet

EXTERNAL CSS -

```
/* External CSS for Blog Styling */
main {
  background-color: white;
  padding: 20px;
  border-radius: 10px;
  box-shadow: 0 0 10px rgba(0,0,0,0.1);
}
h2 {
  color: #007bff;
}
ul, ol {
  margin-left: 20px;
}
a {
  color: #28a745;
  text-decoration: none;
}
a:hover {
  text-decoration: underline;
  color: #155724;
}
img {
  display: block;
  margin: 0 auto 20px auto;
  border-radius: 10px;
}
```

Figure 15 - Code Snippet

OUTPUT-

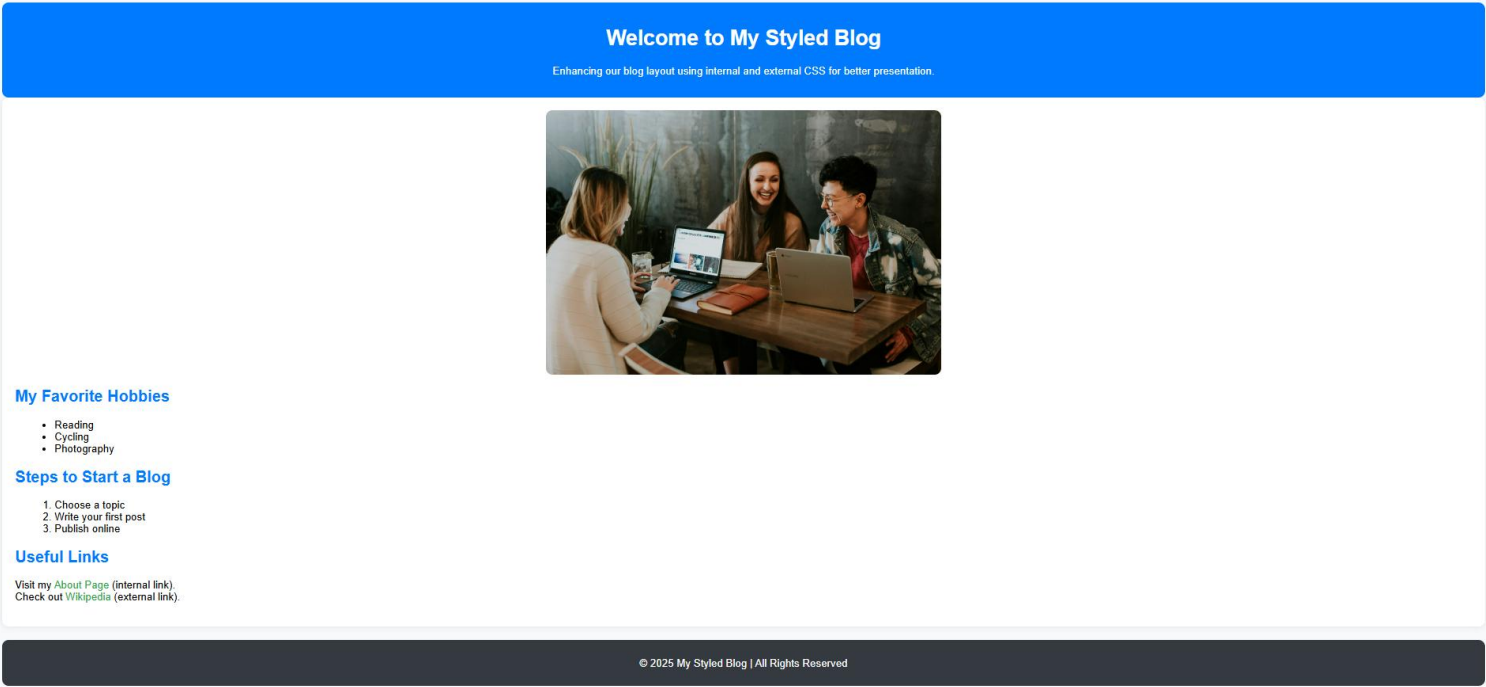


Figure 16 - Final Web Page Layout

EXPERIMENT – 09

AIM - Utilize class selectors, ID selectors, and element selectors to style specific components of the webpage. Demonstrate CSS rule precedence and specificity by combining inline, internal, and external styles in one project

In this experiment, we explored how different **CSS selectors** and **types of styles** interact to determine which rules take precedence when multiple styles target the same element.

1. Types of CSS Selectors:

- **Element Selector** – Targets HTML elements directly (e.g., p, h1, div).
- **Class Selector** – Targets elements using the class attribute (e.g., .highlight).
- **ID Selector** – Targets a specific element using its unique id (e.g., #main-title).

2. CSS Specificity and Precedence:

When multiple CSS rules apply to the same element:

1. **Inline CSS** (inside an element's style attribute) has the **highest priority**.
2. **ID selectors** override **class** and **element selectors**.
3. **Class selectors** override **element selectors**.
4. **External CSS** can be overridden by **internal** or **inline** styles if both apply to the same element.

By combining **inline**, **internal**, and **external** styles, we can observe how CSS decides which rule to apply — demonstrating **specificity** and **precedence**.

PROGRAM-

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS Selectors & Specificity</title>
  <!-- External CSS -->
  <link rel="stylesheet" href="style2.css">

  <!-- Internal CSS -->
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f0f0f0;
      margin: 20px;
    }

    header {
      background-color: #007bff;
      color: white;
      padding: 15px;
      text-align: center;
      border-radius: 10px;
    }

    /* Element selector */
    p {
```

```

    color: #333;
    font-size: 16px;
}

/* Class selector */
.highlight {
    background-color: yellow;
    color: black;
    padding: 5px;
}

/* ID selector */
#special-para {
    color: red;
    font-weight: bold;
}
</style>
</head>

<body>
  <header>
    <h1 id="main-title">Understanding CSS Specificity</h1>
  </header>

  <main>
    <h2 class="highlight">Inline, Internal, and External Styles</h2>
    <p>This paragraph is styled using <strong>element selectors</strong> from internal CSS.</p>

    <p id="special-para">This paragraph uses an <strong>ID selector</strong>, which overrides
the element selector.</p>

    <p class="highlight" style="color: blue; font-weight: bold;">
      This paragraph demonstrates <strong>inline CSS</strong>, which overrides both class and ID
styles.
    </p>
  </main>

  <footer>
    <p>&copy; 2025 CSS Experiment | All Rights Reserved</p>
  </footer>
</body>
</html>

```

Figure 17 - Code Snippet

EXTERNAL CSS –

```
/* External CSS */
h2 {
  color: green;
  text-decoration: underline;
}

p {
  color: gray;
}

/* Class selector in external CSS */
.highlight {
  background-color: lightblue;
  color: darkblue;
}

/* ID selector in external CSS */
#special-para {
  color: purple;
}
```

Figure 18 - Code Snippet

OUTPUT-

Understanding CSS Specificity

Inline, Internal, and External Styles

This paragraph is styled using **element selectors** from internal CSS.

This paragraph uses an **ID selector**, which overrides the element selector.

This paragraph demonstrates **inline CSS**, which overrides both class and ID styles.

© 2025 CSS Experiment | All Rights Reserved

Figure 19 - Final Web Page Layout

EXPERIMENT – 10

AIM - Use CSS to apply text formatting, background colors, borders, padding, and margins to different webpage elements

In this experiment, we learned how to enhance the **appearance and spacing** of webpage elements using various **CSS properties**.

1. **Text Formatting** – Controls how text appears. Common properties include:
 - font-family – defines the font type.
 - font-size – sets text size.
 - font-weight – makes text bold.
 - color, text-align, and text-decoration – adjust color, alignment, and decoration.
2. **Background Colors** – The background-color property adds color behind an element, improving contrast and readability.
3. **Borders** – The border property defines the line around elements. You can change its width, style, and color.
4. **Padding** – Creates space **inside** the element, between content and border.
5. **Margins** – Creates space **outside** the element, between it and surrounding elements.

By combining these properties, we can make webpages more organized, readable, and visually appealing.

PROGRAM-

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS Formatting Example</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f5f5f5;
      margin: 30px;
    }

    header {
      background-color: #007bff;
      color: white;
      text-align: center;
      padding: 20px;
      border-radius: 10px;
    }

    h2 {
      color: #333;
      text-decoration: underline;
    }

    .intro {
      background-color: #e3f2fd;
      border: 2px solid #90caf9;
      padding: 15px;
```

```

margin: 15px 0;
border-radius: 8px;
}

.highlight {
background-color: #fff3cd;
border-left: 5px solid #ffc107;
padding: 10px;
margin: 10px 0;
font-style: italic;
}

.content {
background-color: white;
padding: 20px;
border: 1px solid #ccc;
margin-top: 20px;
border-radius: 8px;
line-height: 1.6;
}

footer {
background-color: #343a40;
color: white;
text-align: center;
padding: 10px;
border-radius: 8px;
margin-top: 20px;
}
</style>
</head>

<body>
  <header>
    <h1>Formatting with CSS</h1>
    <p>Demonstrating text styles, backgrounds, borders, padding, and margins.</p>
  </header>

  <main>
    <div class="intro">
      <h2>Introduction</h2>
      <p>
        CSS allows us to control spacing, colors, and font styles to create a visually appealing layout.
      </p>
    </div>

    <div class="highlight">
      Tip: Use padding for internal spacing and margins for external spacing.
    </div>

    <div class="content">
      <h2>Example Content</h2>

```

```
<p>
  This section showcases a white content box with padding, margin, and a soft border.
  Text inside this box uses default paragraph styling, with improved readability due to
spacing and line height.
</p>
</div>
</main>

<footer>
  <p>&copy; 2025 CSS Styling Demo | All Rights Reserved</p>
</footer>
</body>
</html>
```

Figure 20 - Code Snippet

OUTPUT-

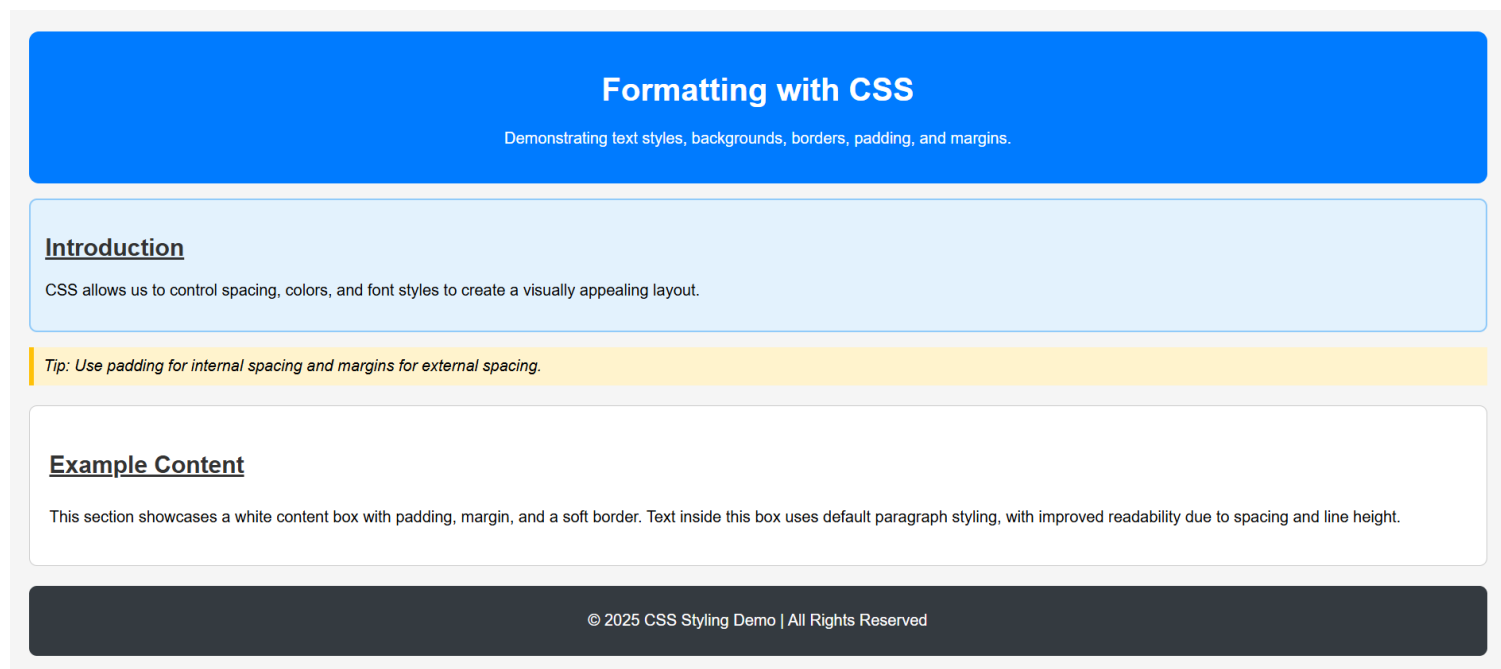


Figure 21 - Final Web Page Layout

EXPERIMENT – 11

AIM - Illustrate the CSS box model by styling elements with borders and adjusting their spacing properties

The **CSS Box Model** is the foundation of web page layout. Every HTML element is considered a **rectangular box**, which consists of the following four areas:

1. **Content** – The actual text, image, or other content inside the element.
2. **Padding** – The space between the content and the element's border.
3. **Border** – A visible line surrounding the padding and content.
4. **Margin** – The space outside the border, separating the element from neighbouring elements.

By adjusting these properties, we can control how much space elements occupy and how they are positioned relative to one another.

PROGRAM-

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS Box Model Example</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f8f9fa;
      margin: 30px;
    }

    header {
      text-align: center;
      background-color: #007bff;
      color: white;
      padding: 15px;
      border-radius: 8px;
    }

    /* Flex container to align boxes side by side */
    .box-container {
      display: flex;
      justify-content: space-between;
      flex-wrap: wrap;
      margin-top: 30px;
    }
    .box {
      background-color: #ffffff;
      border: 3px solid #007bff;
      padding: 20px;
      margin: 10px;
      width: 45%;
      box-shadow: 0 0 8px rgba(0, 0, 0, 0.1);
      border-radius: 8px;
    }
  </style>
</head>
<body>
  <header>
    <h1>CSS Box Model</h1>
  </header>
  <div class="box-container">
    <div class="box">
      <p>Box 1</p>
    </div>
    <div class="box">
      <p>Box 2</p>
    </div>
  </div>
</body>
</html>
```

```

.inner-box {
  background-color: #e3f2fd;
  border: 2px dashed #64b5f6;
  padding: 15px;
  margin: 10px;
}

footer {
  text-align: center;
  background-color: #343a40;
  color: white;
  padding: 10px;
  border-radius: 8px;
  margin-top: 30px;
}
</style>
</head>

<body>
  <header>
    <h1>Understanding the CSS Box Model</h1>
    <p>Each element can have content, padding, border, and margin.</p>
  </header>

  <main>
    <div class="box-container">
      <!-- Left Box -->
      <div class="box">
        <h2>Outer Box 1</h2>
        <p>This outer box has a solid border, padding, and margin. It represents one section of content.</p>

        <div class="inner-box">
          <h3>Inner Box</h3>
          <p>This inner box has its own padding, margin, and border – nested inside the outer box.</p>
        </div>
      </div>

      <!-- Right Box -->
      <div class="box">
        <h2>Outer Box 2</h2>
        <p>This is the second box placed side by side . It has similar padding and borders to maintain symmetry.</p>

        <div class="inner-box">
          <h3>Inner Box</h3>
          <p>This inner box helps demonstrate consistent spacing and alignment across multiple elements.</p>
        </div>
      </div>
    </div>
  </main>

```

```
</main>

<footer>
  <p>&copy; 2025 Box Model Demo | All Rights Reserved</p>
</footer>
</body>
</html>
```

Figure 22 - Code Snippet

OUTPUT-

Understanding the CSS Box Model

Each element can have content, padding, border, and margin.

Outer Box 1

This outer box has a solid border, padding, and margin. It represents one section of content.

Inner Box

This inner box has its own padding, margin, and border — nested inside the outer box.

Outer Box 2

This is the second box placed side by side . It has similar padding and borders to maintain symmetry.

Inner Box

This inner box helps demonstrate consistent spacing and alignment across multiple elements.

© 2025 Box Model Demo | All Rights Reserved

Figure 23 - Final Web Page Layout

EXPERIMENT – 12

AIM – Implement a variety of CSS units such as em, rem, %, and px to observe how different units impact design

In CSS, measurements are defined using different **units** that control the **size**, **spacing**, and **layout** of elements. Each unit behaves differently depending on whether it's **relative** (scales with context) or **absolute** (fixed).

1. px (Pixels)

- An **absolute unit**.
- Defines a fixed size that does **not scale** with screen or font settings.
- Example: font-size: 16px;

2. % (Percentage)

- A **relative unit**.
- Defines a size relative to the **parent element's** dimensions.
- Example: width: 50%; makes the element half the width of its container.

3. em (Element-relative)

- A **relative unit** based on the **font size of the parent**.
- Example: 2em means twice the parent's font size.

4. rem (Root-relative)

- A **relative unit** based on the **root (html)** font size, not the parent.
- More predictable than em because it's independent of nested elements.

By using a combination of these units, we can create designs that are **flexible**, **scalable**, and **responsive** across devices.

PROGRAM-

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS Units Demonstration</title>
  <style>
    /* Base font size for rem calculation */
    html {
      font-size: 16px;
    }

    body {
      font-family: Arial, sans-serif;
      background-color: #f8f9fa;
      margin: 20px;
    }

    header {
      background-color: #007bff;
```

```
    color: white;
    padding: 1rem; /* rem = root-relative */
    text-align: center;
    border-radius: 8px;
}

.container {
    display: flex;
    justify-content: space-between;
    flex-wrap: wrap;
    margin-top: 2rem;
}

.box {
    background-color: #ffffff;
    border: 2px solid #007bff;
    padding: 1em; /* em = parent-relative */
    margin: 1em;
    width: 45%; /* percentage-based width */
    box-shadow: 0 0 8px rgba(0, 0, 0, 0.1);
    border-radius: 8px;
}

.box h2 {
    font-size: 1.5rem; /* root-relative */
    margin-bottom: 0.5em;
}

.px-example {
    font-size: 20px; /* fixed size */
    color: #343a40;
}

.em-example {
    font-size: 1.5em; /* relative to parent font size */
    color: #28a745;
}

.rem-example {
    font-size: 1.5rem; /* relative to root font size */
    color: #d63384;
}

.percent-example {
    font-size: 120%; /* relative to parent font size */
    color: #fd7e14;
}

footer {
    background-color: #343a40;
    color: white;
    text-align: center;
    padding: 0.8rem;
    border-radius: 8px;
    margin-top: 2rem;
}
```

```
</style>
</head>
<body>
  <header>
    <h1>Exploring CSS Units</h1>
    <p>Understanding px, em, rem, and % units in design</p>
  </header>
  <main class="container">
    <div class="box">
      <h2>Box 1: Fixed Units (px)</h2>
      <p class="px-example">
        This text uses <strong>pixels (px)</strong> for font sizing.
        It stays the same size regardless of screen or parent settings.
      </p>
    </div>
    <div class="box">
      <h2>Box 2: Relative Units (em, rem, %)</h2>
      <p class="em-example">
        This text uses <strong>em</strong> units — it scales with the parent’s font size.
      </p>
      <p class="rem-example">
        This text uses <strong>rem</strong> units — it scales with the root (html) font size.
      </p>
      <p class="percent-example">
        This text uses <strong>%</strong> units — it’s relative to the parent’s font size.
      </p>
    </div>
  </main>
  <footer>
    <p>&copy; 2025 CSS Units | All Rights Reserved</p>
  </footer>
</body>
</html>
```

Figure 24 - Code Snippet

OUTPUT-

Exploring CSS Units

Understanding px, em, rem, and % units in design

Box 1: Fixed Units (px)

This text uses **pixels (px)** for font sizing. It stays the same size regardless of screen or parent settings.

Box 2: Relative Units (em, rem, %)

This text uses **em** units — it scales with the parent’s font size.

This text uses **rem** units — it scales with the root (html) font size.

This text uses **%** units — it’s relative to the parent’s font size.

© 2025 CSS Units | All Rights Reserved

Figure 25 - Final Web Page Layout

EXPERIMENT – 13

AIM - Design a responsive navigation bar using Flexbox properties such as justify-content and align-items

Flexbox (**Flexible Box Layout**) is a CSS layout module that provides a more efficient way to arrange and align elements in a container, even when their size is unknown or dynamic.

It's ideal for building **responsive navigation bars** because it automatically adjusts element positioning for different screen sizes.

Key Flexbox Properties Used:

1. **display: flex;** Activates Flexbox on the container, enabling flexible item alignment.
2. **justify-content:** Aligns items **horizontally** within the container.

Examples:

- space-between → equal spacing between items
- center → items centered horizontally

3. **align-items:** Aligns items **vertically** within the container.
4. **flex-wrap:** Allows items to wrap onto multiple lines when the screen width is reduced, making the layout responsive.

By combining these properties, we can create a **clean, centered, and responsive navigation bar** that adapts to mobile and desktop views.

PROGRAM-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Flexbox Navbar</title>
  <style>
    body {
      margin: 0;
      font-family: Arial, sans-serif;
      background-color: #f8f9fa;
    }

    /* Navigation bar container */
    nav {
      display: flex;
      justify-content: space-between; /* horizontal alignment */
      align-items: center; /* vertical alignment */
      background-color: #6f42c1; /* Updated Purple Theme */
      padding: 15px 30px;
      flex-wrap: wrap; /* make it responsive */
    }
```

```
/* Logo or brand name */
.logo {
  font-size: 1.5rem;
  font-weight: bold;
  color: white;
}

/* Navigation links container */
.nav-links {
  display: flex;
  gap: 20px; /* spacing between links */
}

/* Individual link styles */
.nav-links a {
  color: white;
  text-decoration: none;
  font-size: 1rem;
  transition: color 0.3s ease;
}

.nav-links a:hover {
  color: #d6bbfc; /* Light purple hover */
}

/* Responsive design for smaller screens */
@media (max-width: 600px) {
  nav {
    flex-direction: column;
    align-items: flex-start;
  }

  .nav-links {
    flex-direction: column;
    width: 100%;
    margin-top: 10px;
  }

  .nav-links a {
    padding: 8px 0;
  }
}
</style>
</head>

<body>
  <nav>
    <div class="logo">MyWebsite</div>
    <div class="nav-links">
      <a href="#">Home</a>
      <a href="#">About</a>
      <a href="#">Services</a>
      <a href="#">Contact</a>
    </div>
  </nav>
</body>
```



```
</div>
</nav>

<main style="padding: 20px;">
  <h2>Welcome to My Responsive Page</h2>
  <p>
    This example demonstrates how to design a responsive navigation bar using Flexbox
properties such as
    <strong>justify-content</strong> and <strong>align-items</strong>
  </p>
</main>
</body>
</html>
```

Figure 26- Code Snippet

OUTPUT-

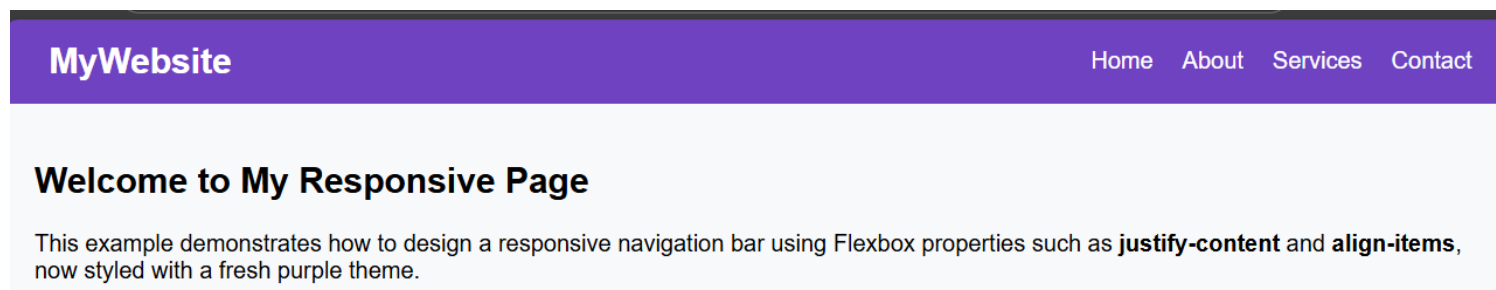


Figure 27 - Final Web Page Layout

EXPERIMENT – 14

AIM - Implement media queries to ensure the layout adapts correctly on different screen sizes like mobile and desktop

Media Queries allow CSS to apply styles based on device properties like width, height, or orientation. They make a webpage **responsive**, ensuring it looks good on all screens.

PROGRAM-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Layout with Media Queries</title>
  <style>
    body {
      margin: 0;
      font-family: Arial, sans-serif;
      background-color: #fafafa;
    }

    header {
      background-color: #6f42c1;
      color: white;
      text-align: center;
      padding: 20px;
    }

    .content {
      display: flex;
      justify-content: space-around;
      padding: 20px;
    }

    .box {
      background: #d6bbfc;
      padding: 30px;
      border-radius: 8px;
      width: 30%;
      text-align: center;
    }

    /* Responsive behavior */
    @media (max-width: 768px) {
      .content {
        flex-direction: column;
        align-items: center;
      }

      .box {
        width: 80%;
      }
    }
  </style>

```

```

        margin-bottom: 15px;
    }
}
</style>
</head>
<body>
    <header>
        <h1>Responsive Layout Example</h1>
    </header>

    <section class="content">
        <div class="box">Box 1</div>
        <div class="box">Box 2</div>
        <div class="box">Box 3</div>
    </section>
</body>
</html>

```

Figure 28 - Code Snippet

OUTPUT-

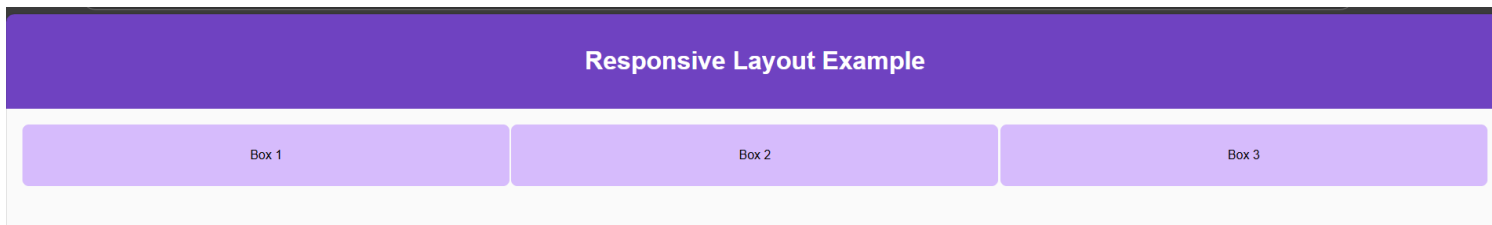


Figure 29 - Final Web Page Layout on Desktop

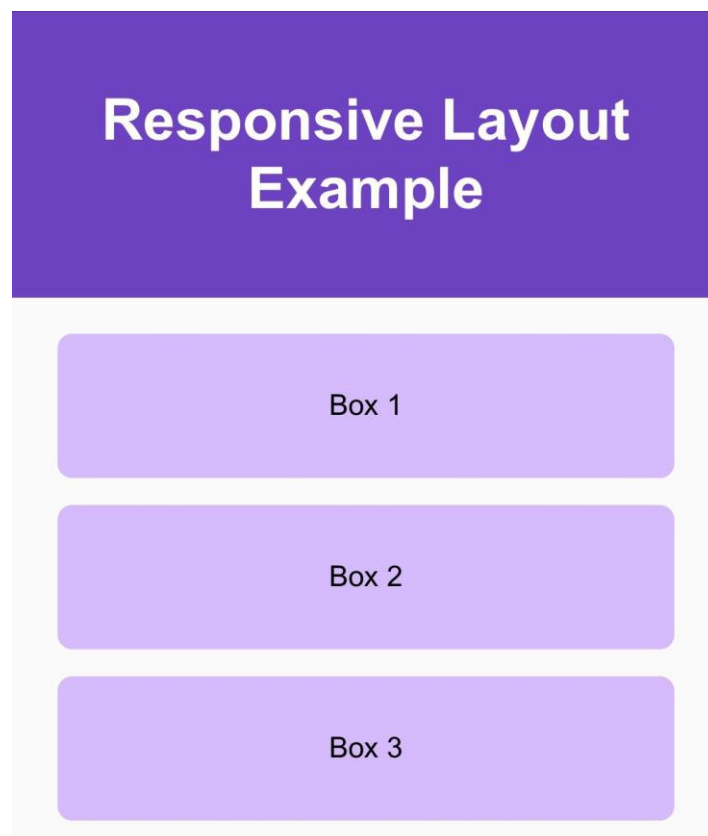


Figure 30 - Final Web Page Layout on Mobile

EXPERIMENT – 15

AIM - Apply responsive design using min-width, max-width, and the viewport meta tag for mobile optimization.

The <meta name="viewport"> tag controls the page's width and scaling on mobile devices. The min-width and max-width properties restrict element resizing to prevent overflow or distortion.

PROGRAM-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Design Example</title>
  <style>
    body {
      margin: 0;
      font-family: sans-serif;
      background-color: #f3f3f3;
      text-align: center;
    }

    img {
      width: 80%;
      max-width: 600px;
      min-width: 200px;
      border-radius: 10px;
      margin: 20px;
    }

    p {
      font-size: 1rem;
      max-width: 800px;
      margin: auto;
      padding: 10px;
    }
  </style>
</head>
<body>
  <h1>Responsive Image & Text Example</h1>
  
  <p>This image and text adjust size automatically using <strong>min-width</strong>,
  <strong>max-width</strong>, and the <strong>viewport meta tag</strong> for better readability on
  all devices.</p>
</body>
</html>
```

Figure 31 - Code Snippet

Responsive Image & Text Example



This image and text adjust size automatically using **min-width**, **max-width**, and the **viewport meta tag** for better readability on all devices.

Figure 32 - Final Web Page Layout

EXPERIMENT – 16

AIM - Use CSS pseudo-classes like :hover and :active to create interactive visual effects.

Pseudo-classes define a special state of an element.

- **:hover** → when the mouse is over the element
- **:active** → when the element is clicked

PROGRAM-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Hover and Active Effects</title>
  <style>
    button {
      background-color: #6f42c1;
      color: white;
      border: none;
      padding: 15px 25px;
      font-size: 1rem;
      border-radius: 8px;
      transition: 0.3s;
    }
    button:hover {
      background-color: #9b59b6;
      transform: scale(1.1);
    }
    button:active {
      background-color: #4b0082;
      transform: scale(0.95);
    }
  </style>
</head>
<body style="text-align:center; padding:50px;">
  <button>Hover or Click Me</button>
</body>
</html>
```

Figure 33 - Code Snippet

OUTPUT-

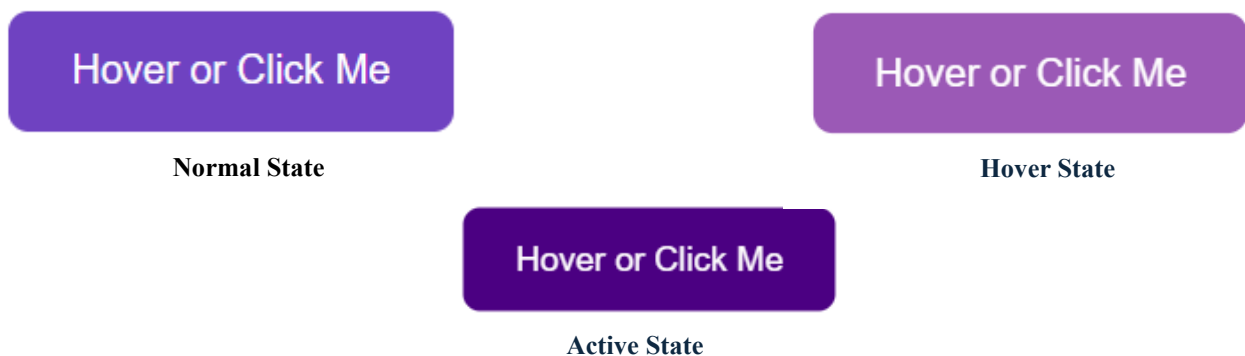


Figure 34 - Final Web Page Layout

EXPERIMENT – 17

AIM - Use pseudo-elements `::before` and `::after` to add decorative or functional content without modifying HTML.

Pseudo-elements let us insert virtual elements into the DOM for decoration or emphasis.

PROGRAM-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Pseudo Elements Example</title>
  <style>
    h1 {
      position: relative;
      text-align: center;
      color: #6f42c1;
    }

    h1::before {
      content: "★ ";
      color: gold;
    }

    h1::after {
      content: " ★";
      color: gold;
    }

  </style>
</head>
<body>
  <h1>Welcome to My Page</h1>
</body>
</html>
```

Figure 35 - Code Snippet

OUTPUT-

★ Welcome to My Page ★

Figure 36 - Final Web Page Layout

EXPERIMENT – 18

AIM - Develop a simple CSS animation such as fading in/out an image or transforming a button on hover.

CSS animations use @keyframes to define stages of an animation.

Properties like transform, opacity, and background-color can smoothly transition between states.

PROGRAM-

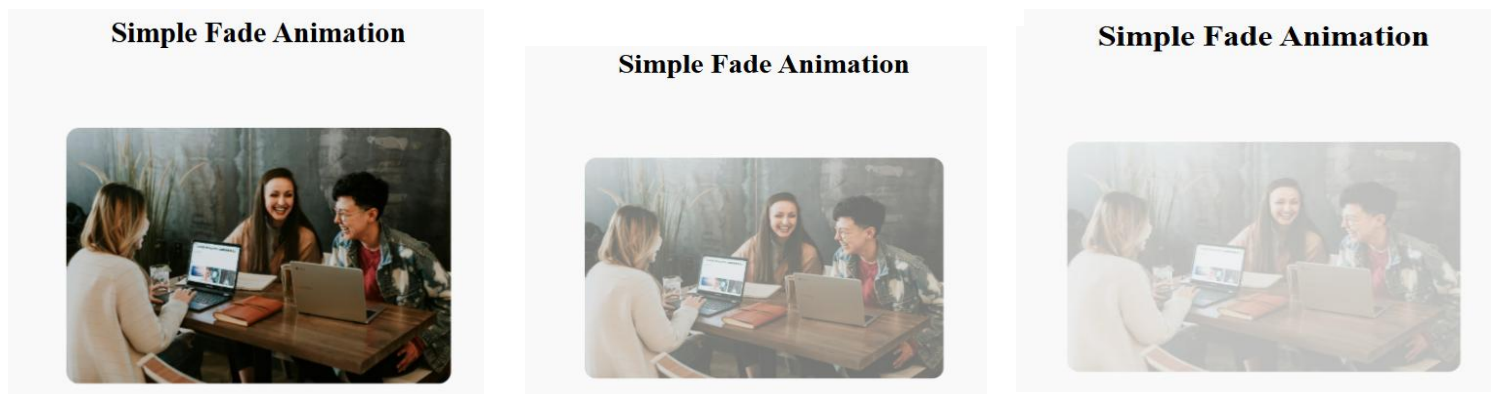
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>CSS Animation Example</title>
  <style>
    .fade {
      width: 300px;
      margin: 50px auto;
      border-radius: 10px;
      animation: fadeIn 3s infinite alternate;
    }

    @keyframes fadeIn {
      from {
        opacity: 0.2;
        transform: scale(0.95);
      }
      to {
        opacity: 1;
        transform: scale(1.05);
      }
    }
  </style>
</head>
<body style="background-color:#f9f9f9; text-align:center;">
  <h2>Simple Fade Animation</h2>
  
</body>
</html>
```

Figure 37 - Code Snippet

OUTPUT-

Figure 38 - Final Web Page Layout



EXPERIMENT – 19

AIM - Develop a Web Page Layout using Bootstrap Grid System and Utility Classes

Bootstrap is a popular front-end framework that simplifies responsive web design using its built-in **grid system** and **utility classes**.

Key Concepts:

- The **grid system** divides the page into **12 equal columns**.
- **Rows (.row)** contain **columns (.col)**, which automatically adjust to screen size.
- **Utility classes** (like p-3, text-center, bg-light) apply quick styling without writing custom CSS.

Responsive classes example :

- **.col-sm-*** – for small devices (≥ 576 px)
- **.col-md-*** – for medium devices (≥ 768 px)
- **.col-lg-*** – for large devices (≥ 992 px)

PROGRAM-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Bootstrap Grid Layout</title>
  <!-- Bootstrap CDN -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
  <header class="bg-primary text-white text-center p-4">
    <h1>My Bootstrap Web Layout</h1>
  </header>

  <div class="container mt-4">
    <div class="row text-center">
      <div class="col-md-4 col-sm-12 p-3 bg-light border">
        <h3>Column 1</h3>
        <p>This section contains sample content using Bootstrap grid.</p>
      </div>
      <div class="col-md-4 col-sm-12 p-3 bg-secondary text-white border">
        <h3>Column 2</h3>
        <p>Bootstrap utility classes manage padding, color, and alignment.</p>
      </div>
      <div class="col-md-4 col-sm-12 p-3 bg-light border">
        <h3>Column 3</h3>
        <p>The layout adjusts automatically for all screen sizes.</p>
      </div>
    </div>
  </div>
```

```
</div>

<footer class="bg-dark text-white text-center p-3 mt-4">
  <p>&copy; 2025 My Website | Designed with Bootstrap</p>
</footer>
</body>
</html>
```

Figure 39 - Code Snippet

OUTPUT-



Figure 40 - Final Web Page Layout

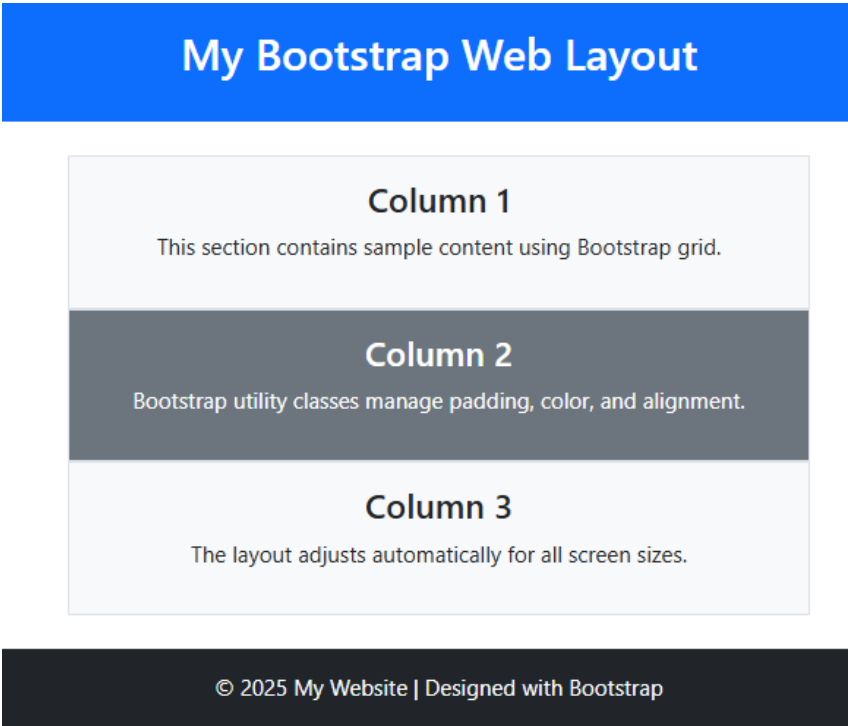


Figure 41 - Final Web Page Layout (Mobile)

EXPERIMENT – 20

AIM - Design and deploy a Static Website to GitHub Pages

GitHub Pages is a free web-hosting service provided by **GitHub** that allows developers to publish static websites directly from a GitHub repository.

A *static website* is composed of fixed files such as **HTML**, **CSS**, and **JavaScript** that are served to users without the need for a backend server or database.

To deploy a website using GitHub Pages:

1. Create or sign in to a **GitHub account**.
2. Create a new **repository** and name it username.github.io.
3. Upload all project files (HTML, CSS, images, etc.) to the repository.
4. Go to **Settings** → **Pages**, select the **main branch**, and save the configuration.
5. After a few moments, the website becomes publicly available at <https://username.github.io>.

GitHub Pages is an efficient platform for hosting **portfolios**, **project demos**, and **personal websites**, since it offers version control, free hosting, and automatic HTTPS security.

This experiment demonstrates practical knowledge of how to **design, organize, and publish** a complete static website using modern version-control and deployment tools.

PROGRAM-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>UI Showroom</title>
  <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;500;600&display=swap"
rel="stylesheet">
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body {
      font-family: 'Poppins', sans-serif;
      background: #f8f9fa;
      color: #212529;
      padding: 30px;
      line-height: 1.6;
    }
    .container {
      max-width: 800px;
      margin: 0 auto;
    }
    header {
      text-align: center;
      margin-bottom: 40px;
      padding: 20px 0;
    }
  </style>
</head>
<body>
  <div class="container">
    <div class="header">
      <h1>UI Showroom</h1>
    </div>
  </div>
</body>
</html>
```

```
h1 {
  font-size: 2rem;
  font-weight: 600;
  margin-bottom: 10px;
}

.subtitle {
  color: #6c757d;
  font-size: 1rem;
}

.section {
  background: white;
  border-radius: 12px;
  padding: 25px;
  margin-bottom: 30px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.08);
}

.section-title {
  font-size: 1.4rem;
  margin-bottom: 20px;
  padding-bottom: 12px;
  border-bottom: 1px solid #e9ecef;
  font-weight: 600;
}

/* Buttons */
.btn {
  padding: 10px 24px;
  border-radius: 50px;
  font-weight: 600;
  display: inline-block;
  margin: 0 10px 10px 0;
  text-decoration: none;
  transition: all 0.2s;
}

.btn-primary {
  background: #4361ee;
  color: white;
  border: 2px solid #4361ee;
}

.btn-outline {
  background: transparent;
  color: #4361ee;
  border: 2px solid #4361ee;
}

.btn-ghost {
  background: transparent;
  color: #6c757d;
  border: 2px solid transparent;
}

.btn:hover {
  transform: translateY(-2px);
  box-shadow: 0 4px 8px rgba(0,0,0,0.1);
}
```

```
.btn-primary:hover { background: #3a56d4; }
.btn-outline:hover { background: rgba(67,97,238,0.08); }
.btn-ghost:hover { background: #f8f9fa; color: #212529; border-color: #dee2e6; }

/* Cards */
.cards {
  display: flex;
  gap: 20px;
  flex-wrap: wrap;
  margin-top: 15px;
}
.card {
  background: white;
  border-radius: 12px;
  padding: 20px;
  width: calc(50% - 10px);
  box-shadow: 0 2px 8px rgba(0,0,0,0.08);
  transition: transform 0.2s;
}
.card:hover {
  transform: translateY(-3px);
  box-shadow: 0 4px 12px rgba(0,0,0,0.12);
}
.card h3 {
  margin: 10px 0;
  font-size: 1.1rem;
}
.card p {
  color: #6c757d;
  font-size: 0.95rem;
}

/* Toggle */
.toggle {
  display: flex;
  align-items: center;
  margin-top: 15px;
}
.toggle-item {
  display: flex;
  align-items: center;
  margin-right: 30px;
}
.toggle-label {
  margin-right: 12px;
  font-weight: 500;
}
.switch {
  position: relative;
  display: inline-block;
  width: 50px;
  height: 26px;
}
```

```
.switch input {
  opacity: 0;
  width: 0;
  height: 0;
}
.slider {
  position: absolute;
  cursor: pointer;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: #e9ecef;
  border-radius: 34px;
  transition: .2s;
}
.slider:before {
  position: absolute;
  content: "";
  height: 18px;
  width: 18px;
  left: 4px;
  bottom: 4px;
  background: white;
  border-radius: 50%;
  transition: .2s;
}
input:checked + .slider {
  background: #4361ee;
}
input:checked + .slider:before {
  transform: translateX(24px);
}

/* Form */
.form {
  margin-top: 15px;
}
.form-group {
  margin-bottom: 18px;
}
.form-group label {
  display: block;
  margin-bottom: 6px;
  font-weight: 500;
}
.form-control {
  width: 100%;
  padding: 12px;
  border: 1px solid #dee2e6;
  border-radius: 8px;
  font-family: 'Poppins', sans-serif;
}
```

```

.form-control:focus {
  outline: none;
  border-color: #4361ee;
  box-shadow: 0 0 0 3px rgba(67,97,238,0.2);
}
textarea.form-control {
  min-height: 100px;
  resize: vertical;
}

/* Responsive */
@media (max-width: 600px) {
  .card { width: 100%; }
  .btn { display: block; margin: 0 0 10px 0; width: 100%; }
  .toggle-item { margin-right: 20px; }
}
</style>
</head>
<body>
  <div class="container">
    <header>
      <h1>UI Showroom</h1>
      <p class="subtitle">Minimal UI components with pure HTML & CSS</p>
    </header>

    <section class="section">
      <h2 class="section-title">Buttons</h2>
      <a href="#" class="btn btn-primary">Primary</a>
      <a href="#" class="btn btn-outline">Outline</a>
      <a href="#" class="btn btn-ghost">Ghost</a>
    </section>

    <section class="section">
      <h2 class="section-title">Cards</h2>
      <div class="cards">
        <div class="card">
          <h3>Card Title</h3>
          <p>Simple shadow card with hover effect</p>
        </div>
        <div class="card">
          <h3>Another Card</h3>
          <p>Clean design with subtle elevation</p>
        </div>
      </div>
    </section>

    <section class="section">
      <h2 class="section-title">Toggle Switch</h2>
      <div class="toggle">
        <div class="toggle-item">
          <span class="toggle-label">Enable</span>
          <label class="switch">
            <input type="checkbox">
            <span class="slider"></span>
          </label>
        </div>
      </div>
    </section>
  </div>

```

```

    </label>
  </div>
  <div class="toggle-item">
    <span class="toggle-label">Active</span>
    <label class="switch">
      <input type="checkbox" checked>
      <span class="slider"></span>
    </label>
  </div>
</div>
</section>

<section class="section">
  <h2 class="section-title">Contact Form</h2>
  <form class="form">
    <div class="form-group">
      <label for="name">Name</label>
      <input type="text" id="name" class="form-control" placeholder="Your name">
    </div>
    <div class="form-group">
      <label for="email">Email</label>
      <input type="email" id="email" class="form-control" placeholder="Your email">
    </div>
    <div class="form-group">
      <label for="message">Message</label>
      <textarea id="message" class="form-control" placeholder="Your message"></textarea>
    </div>
    <button type="submit" class="btn btn-primary">Submit</button>
  </form>
</section>
</div>
</body>
</html>

```

Figure 42 - Code Snippet

OUTPUT-

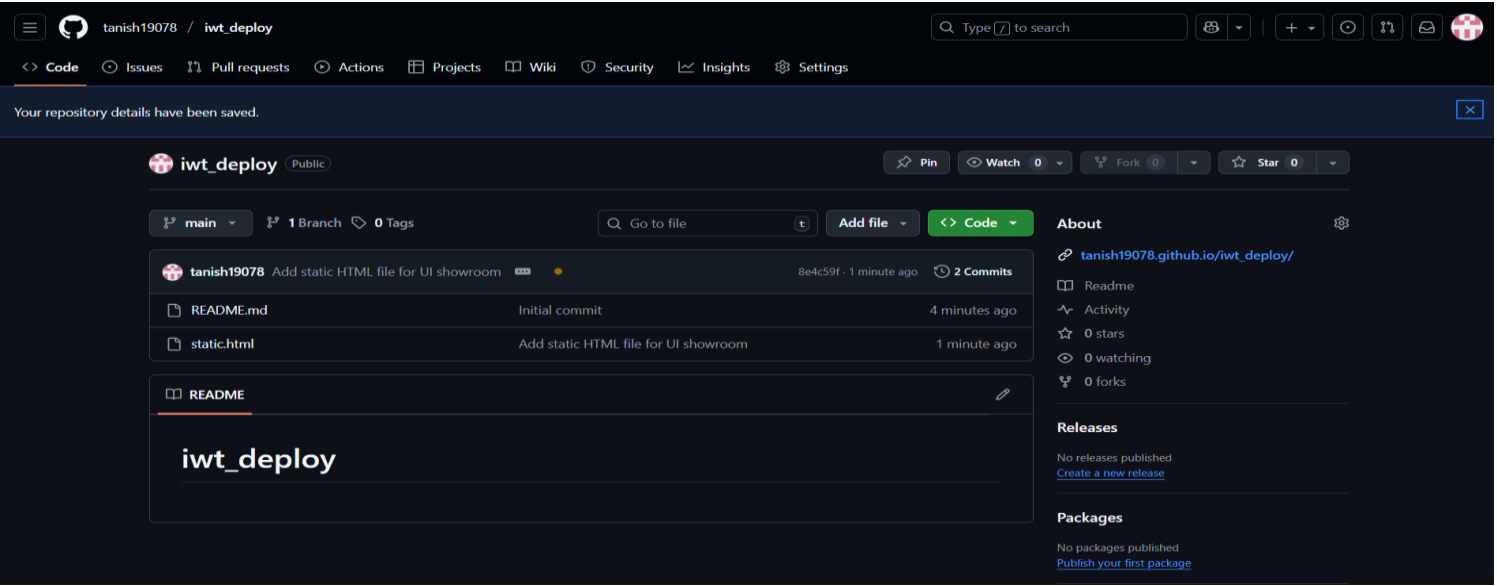


Figure 43 – Github Deployment

UI Showroom

Minimal UI components with pure HTML & CSS

Buttons

Primary

Outline

Ghost

Cards

Card Title

Simple shadow card with hover effect

Another Card

Clean design with subtle elevation

Toggle Switch

Enable



Active



Contact Form

Name

Your name

Email

Your email

Message

Your message



Submit

Figure 44 – Final WebPage Layout